Atividade Avaliativa A2 - Engenharia de Software

Guilherme Rozzi Dicati - a2605660
Enzo Livrari - a2552795

Projeto Inicial

No protótipo a gente implementou só a História 1 (reserva de horário de monitoria), que é a de prioridade alta. Tudo foi feito com classes e métodos que os testes chamam direto, sem usar input no terminal.

O código ficou no arquivo monitoria.py, com duas classes. A primeira é a **Horario**, que representa um horário de 30 minutos de monitoria. Ela guarda o id, a disciplina, o nome do monitor, a semana, o dia, a hora, se o atendimento é individual ou em grupo e o status (começa como livre e vira ocupado quando alguém reserva). Também guarda o RA do aluno e o assunto da dúvida.

A segunda é a **SistemaMonitoria**, que tem a lista de horários e as regras do sistema. Nela tem o método cadastrar_horario, que cria um horário livre, mas só se a matéria for uma das quatro do curso (Cálculo, Física, POO ou Estruturas de Dados). Tem também o listar_horarios_livres, que recebe a disciplina e mostra só os horários livres dela.

A reserva é feita pelo método reservar_horario, que recebe o id do horário, o RA e o assunto. Antes de reservar ele confere três coisas: se nenhum campo veio vazio, se o horário existe e está livre (pra dois alunos não pegarem o mesmo horário individual) e se o aluno ainda não tem 2 horários na mesma matéria naquela semana. Se estiver tudo certo, o horário vira ocupado e os dados do aluno são salvos. Se alguma dessas checagens falhar, a reserva não acontece e o método levanta um ValueError com a mensagem do erro.

Os testes de aceitação ficam na classe TestReservaMonitoria, usando unittest. Cada critério tem seu teste:

Critério 1 (reservar horário livre): test_reservar_horario_livre_sucesso.
Critério 2 (bloquear horário individual já ocupado): test_erro_horario_individual_ja_ocupado.
Critério 3 (limite de 2 horários por semana na matéria): test_erro_limite_semanal_atingido e test_permitir_reserva_outra_materia_mesma_semana.
Critério 4 (matéria inválida ou dados em branco): test_erro_disciplina_invalida e test_erro_dados_em_branco.

Relatório de Execução

Como rodar o projeto e os testes

O código é em Python 3 (3.10 ou mais novo) e não precisa instalar nada, porque os testes usam só o unittest, que já vem no Python. Os arquivos estão na pasta codigo/.

Para rodar uma simulação de uso no terminal, é só executar na raiz do repositório:

```
python3 codigo/main.py
```

E para rodar os testes:

```
python3 codigo/test_monitoria.py
```

O que foi planejado x o que foi feito

O cronograma saiu na aula do dia 16/10, quando a gente leu o e-mail da coordenação e dividiu o trabalho. Combinamos que o Guilherme ficaria mais com a documentação e os testes, e o Enzo com o código, e no geral foi assim que aconteceu.

O repositório e as histórias de usuário ficaram prontos no fim de semana (17 e 18/10), e no dia 19 o projeto inicial já estava no GitHub. Com isso o Enzo conseguiu começar o monitoria.py e o main.py no dia 20, e no dia seguinte os testes já estavam escritos em cima do código dele. No dia 22 a gente se juntou pra rodar tudo de novo, arrumar o que faltava e fechar este documento.

Atrasos e mudanças

Não teve atraso. O único ajuste no repositório foi no final: tínhamos subido os arquivos .py direto na raiz e depois criamos a pasta codigo/ e passamos eles pra lá.

Na hora de programar, mudamos algumas coisas pequenas em relação ao projeto inicial:

Na SistemaMonitoria criamos dois métodos auxiliares, buscar_horario e contar_reservas, pra não deixar tudo amontoado dentro do reservar_horario.
Na Horario, em vez de uma string para o RA e outra para o assunto, usamos listas (self.alunos e self.assuntos). Assim a classe já fica pronta para horários em grupo.

Colocamos um teste a mais no critério 1, o test_horario_reservado_sai_da_lista_de_livres, pra conferir que o horário ocupado some da lista de livres.

Documentação Técnica

1. Histórias de usuário

História 1 - Reserva de horário de monitoria
História: Como aluno, quero ver os horários livres de uma disciplina e reservar um atendimento de 30 minutos, pra garantir minha vaga sem bater com o horário de outro aluno.

Prioridade: Alta.

Justificativa: É o que a coordenação mais precisa, porque hoje os horários são marcados por mensagem e vira bagunça. No próprio e-mail eles dizem que, se a reserva funcionar, já resolve boa parte do problema.

Trecho do e-mail: *"A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento. Cada atendimento dura 30 minutos. Não precisa resolver tudo de uma vez. Se a reserva de horários funcionar, já ajuda muito."*

Critérios de aceitação:

Se o horário estiver livre em uma das disciplinas do curso (Cálculo, Física, POO ou Estruturas de Dados), o aluno reserva informando o RA e o assunto da dúvida, e o horário passa a ficar ocupado.
Se o horário individual já estiver ocupado, o sistema não deixa outro aluno reservar e mostra erro de horário indisponível.
O aluno pode reservar no máximo 2 horários por semana na mesma matéria. Se tentar um terceiro, o sistema bloqueia e avisa que o limite da semana foi atingido.

- Se a disciplina não existir ou faltar algum dado (RA, assunto ou horário), a reserva não é feita e o sistema mostra erro de dados inválidos.

História 2 - Cadastro e cancelamento de horários pelo monitor
História: Como monitor, quero cadastrar meus horários do semestre e poder cancelar um dia quando tiver prova, pra que os alunos fiquem sabendo antes de ir até a sala à toa.

Prioridade: Alta.

Justificativa: Sem os horários dos monitores cadastrados, o aluno não tem o que reservar. E o aviso de cancelamento resolve uma reclamação que aparece sempre: aluno indo até a sala e não tendo monitoria.

Trecho do e-mail: *"No início do semestre, cada monitor informa os horários em que pode atender, mas às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."*

Critérios de aceitação:

O monitor cadastra horários de 30 minutos informando dia, hora, matéria e se é individual ou em grupo. O horário entra como livre.

Se o monitor cancelar um horário, mesmo que já tenha aluno marcado, o status vira cancelado e o sistema registra um aviso para o aluno que tinha reservado.

- Horários cancelados não aparecem como disponíveis e não aceitam novas reservas.

História 3 - Relatório mensal de horas dos monitores

História: Como membro da coordenação, quero gerar um relatório por mês com o total de horas de cada monitor, pra controlar o pagamento das bolsas.

Prioridade: Média.

Justificativa: A coordenação precisa disso pra fechar o pagamento do mês, mas só faz sentido depois que os atendimentos já estiverem sendo registrados no sistema.

Trecho do e-mail: *"A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."*

Critérios de aceitação:

Informando o mês, o sistema soma os atendimentos de 30 minutos de cada monitor e mostra o total em horas.
Só a coordenação pode gerar o relatório. Se um aluno ou monitor tentar, o sistema bloqueia com erro de permissão.

- Horários cancelados pelo monitor ou em que o aluno faltou não entram na soma.

História 4 - Consulta de assuntos com mais dúvidas

História: Como professor, quero ver quais conteúdos tiveram mais agendamentos na minha matéria, pra saber onde a turma está com mais dificuldade e revisar em aula.

Prioridade: Baixa.

Justificativa: Ajuda o professor a planejar as aulas, mas não é essencial pro agendamento funcionar e só tem utilidade depois que bastante gente já tiver usado o sistema.

Trecho do e-mail: *"Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."*

Critérios de aceitação:

Quando o professor consulta a matéria, o sistema mostra os assuntos que os alunos colocaram nas reservas, do mais repetido pro menos repetido.
A consulta mostra só a disciplina escolhida, sem misturar com outras matérias.

Se a matéria ainda não tiver nenhuma reserva, o sistema retorna a lista vazia avisando que ainda não há registros.

História: Como monitor, eu quero marcar se o aluno apareceu ou faltou na monitoria agendada para ter controle de quem reserva horário e não vai.

Prioridade: Baixa.
Justificativa: Ajuda a controlar os horários perdidos, mas como os próprios monitores ainda não decidiram se vão bloquear quem falta ou não, fica com prioridade menor que o agendamento em si.
Trecho do e-mail: “Também tem o problema das faltas. Quando o aluno reserva e não aparece, o horário fica perdido. Alguns monitores sugeriram bloquear quem falta muito, mas outros acham isso exagerado.”
Critérios de aceitação:

O monitor consegue marcar “presente” ou “falta” em um atendimento agendado, e o sistema soma +1 no histórico de faltas do aluno caso ele não tenha ido.

Se o aluno acumular 3 faltas no semestre, o sistema mostra um aviso de histórico de faltas para o monitor quando esse aluno fizer uma nova reserva, mas sem bloquear o agendamento por enquanto.

O sistema não deixa lançar presença ou falta em horários que estavam livres, sem ninguém marcado, ou que foram cancelados pelo monitor.

2. Testes de aceitação

Os testes da História 1 estão no arquivo codigo/test_monitoria.py, na classe TestReservaMonitoria. Eles cobrem os 4 critérios, com casos que devem dar certo e casos que devem dar erro.

Critério 1 (reservar horário livre em matéria válida): fizemos dois testes de sucesso. O test_reservar_horario_livre_sucesso reserva o horário 1 de Cálculo com RA e assunto e confere se o status virou ocupado e se os dados foram salvos. O test_horario_reservado_sai_da_lista_de_livres confere se os horários livres de Cálculo caem de 3 para 2 depois da reserva.

Critério 2 (bloquear horário individual ocupado): o test_erro_horario_individual_ja_ocupado reserva o horário 1 para um aluno e depois tenta reservar o mesmo horário com outro RA. O teste confere se o sistema dá ValueError de horário indisponível.

Critério 3 (no máximo 2 horários por semana na mesma matéria): um teste de erro e um de sucesso. O test_erro_limite_semanal_atingido marca 2 horários de Cálculo na semana 1 para o mesmo aluno e tenta um terceiro, esperando ValueError de limite atingido. O test_permitir_reserva_outra_materia_mesma_semana mostra que esse mesmo aluno ainda consegue reservar Física na mesma semana.

Critério 4 (matéria inexistente ou dados em branco): dois testes de erro. O test_erro_disciplina_invalida tenta cadastrar e listar horários de Química e espera ValueError de dados inválidos. O test_erro_dados_em_branco tenta reservar com RA vazio, assunto vazio e id None, confere se todos dão ValueError e se o horário continua livre.

O que ficou sem teste e por quê

Não testamos reserva em grupo (individual=False), porque o critério 2 fala só de conflito em atendimento individual e o e-mail da coordenação não diz quantos alunos cabem num grupo.

Também não testamos o caso de o aluno mandar um texto só com espaços (tipo " "). A validação no código é simples, com "if not", então ela pega só string vazia ("") e None.